Developer Information

Table of Contents

Design **2**
Overview 2
Tax Form Design 3
Tax Tool Design 4
Library Modules **4**
Tax Tables 4
The TaxTable_20XX Classes 4
The TaxTableTmpl Class 5
The TaxTable Module 5
Tax Forms 5
The TaxForm Class 6
The Line Class 6
The TaxFormObj Module 6
The TaxInfo Module 7
The Classes Module 7
Taxpayer 7
The Taxpayer Class 7
The Dependent Class 8
HTML 9
The HTMLBlock Class 9
The HTMLMgr Class 10
The HTMLBuild Class 10
The HTML Module 10
Save and Restore 11
The File Module 11
The SaveRestore Module 11
ToolTips 11
Utilities 12
The Dates Module 12
The Debug Module 12
The Ensure Module 13
The Eval Module 13
The IncludeFile Module 13
The Num Module 13
The Objects Module 14
The SalesTaxFromCDTFA Module 14
The Str Module 14
Tax Tools **15**
Age Calculator 15
Simple Math Calculator 15
Tax Return Calculator 15
Other Tax Tools 15
Software Conventions **16**

Coding Conventions 16
Naming Conventions 16
Design Notes 16
Source Control 17
Repositories 17
Authentication 18
Creating a Token 18
Branches 19
Create the Initial Repository 19
Using Git with the Tax Tools 21
Configuration Settings 21
Create Local Repository 22
Create Account on GitHub 22
Synchronize with Remote Repository 22
Get Status 22
Select a Branch 23
Create a New Branch 23
Add a New File 23
Delete a file 24
Change a File 24
Stage a File 24
Commit a Change 25
Push Changes to the Remote Repository 25
Make a Change to Development Branch 25
Make Change to Main Branch 26
Merge Development Branch into Main Branch 26
Get a Past Version of the Tax Tools 27
Create a Tag 27
Git Help 28
Git Terminology 28
Editing Tools 29
VS Code 29
Installation 29
Configuration Settings 29
Extensions 30
Command Summary 30
Node and NPM 31
Installation 31
Prettier 31
ESLint 32
Git 32
Installation 32

Design

Overview

The basic idea behind the design of the tax tools is that every tax form has the same basic design. That is, every tax form is series of lines, where each line is a step in the calculation of one or a few tax values.

Each line consists of a line number, a description, and a value. The description tells how to calculate the value for that line.

The calculation for any individual line consists of simple operations, such as:

- Reading a value from another form.
- Enter a constant value such as a percentage and dollar amount.
- Performing arithmetic using values determined earlier in the form
- Comparing values to skip steps

Each step calculates the next intermediate value until the last line where the final result is determined. The steps are calculated in order from the first to the last. A step may use values from previous steps or get values from other forms.

If a line requires a more complicated calculation, that calculation is relegated to another tax form or a worksheet, and the current form simply says to get the value from that form. Worksheets are implemented using the same simple design, that is, a series of lines that perform individual steps in a larger calculation.

This design is also highly flexible and extensible. There are probably hundreds of tax forms, but only the tax forms that you intend to use need to be implemented. If a tax form references a value in another form that has not been implemented, it gets a value of 0. If at some point in the future that form is implemented, its value will automatically start to be used.

The calculation is automatic in another way as well. A form will not be calculated until the first time it referenced. Thus, a form will only be calculated if it is referenced by another form and dependencies will be sorted out naturally. The only time this is a problem is if there are forms that either directly or indirectly reference each other. I have only found a few instances of this. For example, calculating the form 1040 triggers the calculation of several other forms, and other forms may reference information on form 1040 (such as AGI). So far, the circular dependencies that I have found can be resolved by fairly simple and isolated workarounds.

Tax Form Design

The simple and consistent design of tax forms allows all tax forms to be implemented in a similar manner. Only the implementation of the individual steps needs to change from one tax form to the next, and this applies to worksheets as well. However, another important aspect of the tax form design is that each form needs to be represented in two formats: HTML and JavaScript.

The HTML version of a tax form is currently only used by the Tax Return Calculator, but it is a capability of all tax forms. The other tools provide a custom web page that shows only the fields from the tax forms that are necessary.

The JavaScript version of a tax form is used by all the tools and is represented by an instance of a JavaScript class (i.e., an object).

Quite a few tax forms and worksheets have already been implemented, and I have found it convenient to lump them into the following categories.

- Input Forms
- Output Forms
- Input Worksheets
- Worksheets

The significance of separating input tax forms and output tax forms, is that input tax forms start on the web page (HTML) and the JavaScript representation of the tax form is created second. With output tax forms, the tax form is calculated first (JavaScript version) and it is then displayed on the web page (HTML version).

Input worksheets are just sections of HTML for allowing the user to enter information that is not found on a tax form. They are not actually tax forms, but they are implemented using the same mechanisms.

When a tax form is implemented, it is not necessary to implement the entire form. One of the features of the tax tools is that it only implements the most commonly used parts of the tax code. For example, in the Tax Return Calculator tool, most or all of the fields are shown when am input tax form is displayed on the web page, but only the fields displayed in green are used in the calculations.

Tax Tool Design

All of the tax tools are based on the idea of providing the result from a small section of the overall income tax calculation. This, in turn, can be accomplished by performing the calculation from one or a few tax forms. For example, the tool to determine how much of the taxpayer's Social Security benefit is taxable only needed to display the result from calculating the Social Security Benefits Worksheet (see 1040 Instructions, lines 6a and 6b). The following steps outline how this was done for this tool.

1. Look at the worksheet to see what information is used in the calculation (that is, find all the lines in the worksheet that reference a value in another tax form).
2. Provide a user interface to get that information from the taxpayer.
3. Create skeleton tax forms that contain only the information provided by the taxpayer.
4. Get the value from the final step in the Social Security Benefits Worksheet. Accessing this value, will trigger the calculation of the worksheet. The steps in the worksheet will access the values in the skeleton tax forms and calculate the result.
5. Display the result from the worksheet to the taxpayer.

A variety of tools can be created that provide tax calculations based on a small set of values provided by the taxpayer. By letting the taxpayer manipulate the information used in the calculation, very useful tools can be created for analyzing various tax situations.

Library Modules

Note: "Modules" is a general term that may refer to JavaScript modules or classes.

The JavaScript modules (not classes) in the tax tools export their functions is a way that requires the module name to prefix the function name when the function is called. This helps keep tract of the myriad utility functions that are part of the tax tools.

The source code for the tax tools is divided into separate folders, one for each tool. However, there is also one folder, `Library`, that contains many modules that are shared by all the tools. This section describes the modules that are in the `Library` folder.

Tax Tables

The TaxTable_20XX Classes

The tax code contains many numeric values that change from year to year. Most of these values are dollar amounts that are adjusted each year for changes in the cost of living. Any of these values that are used by the tax tools are stored in one of the modules listed below. There is one file for each tax year that is supported by the tax tools. Currently, these files are:

- TaxTable_2024.js
- TaxTable_2025.js
- TaxTable_2025.js

The TaxTableTmpl Class

The TaxTableTmpl class contains a common set of methods that are used to access whichever tax table is currently being used. Therefore, a tax table is an instance of the TaxTable_20XX class from one of the above files and that class extends the `TaxTableTmpl` class. This creates an object with the tax information from that year and it has the following methods.

- get_AMT_Exemption() Get AMT exemption amount
- get_AMT_Tax() Get AMT tax given AMT income
- get_CA_IncomeTax() Get California income tax given the income
- getBusinessMileageDeduction() Get the deduction given the miles driven
- getCharitableMileageDeduction() Get the deduction given the miles driven
- getIncomeTaxFromTable() Get the federal income tax given the income
- getMaxLTC() Get the maximum LTC deduction given the age
- getMedicalMileageDeduction() Get the deduction given the miles driven
- getRetirementSavingsPhaseOut() Get the start of the retirement saving credit phase out
- getRMDPeriod() Get number of years to distribute RMD
- getSalesTaxDeduction() Get sales tax deduction given income and family size
- getStandardDeduction() Get the standard deduction
- getTaxValue() Get a simple tax value (table lookup, no calculation)
- getTaxYear() Get the tax year being used for the calculations

The TaxTable Module

The TaxTable module is a separate module that manages the tax tables. It contains the following functions.

- getTaxTable() Get or set the current tax tables
- getTaxYear() Get the tax year being used for the calculations
- reset() Reset the tax tables to the initial state
- setTaxTable() Set the tax tables to a previously saved value

The tax table is initialized by calling `TaxTable.getTaxTable()`. The first time this function is called, it should be passed the tax year you want to use. From then on, this function will return a reference to the same tax table object.

Tax Forms

Note: “Tax forms” is a general term that may refer to tax forms or worksheets or, in some cases, input worksheets.

Tax forms are created as an instance of the TaxForm class. As mentioned before, tax forms have an internal and an external representation. A tax form created from the TaxForm class is the internal representation of the tax form. This is where the tax calculations are performed.

Every tax form is implemented as a separate JavaScript class that extends the TaxForm class. By convention, a class name is capitalized using camel case (first letter of each word is capitalized, no space or character between words) and it must begin with a letter, so for example, “F1040” is the name of the class for form 1040. And, a variable that contain a reference to a tax form object uses the tax form name in snake case (all lower case with words separated by underscores), for example “f1040”.

The main property of the tax form is the `lines` object, which is an object with one instance for each line on the tax form. The lines property is implemented as an object rather than an array, because lines numbers are strings, not numbers (e.g., "`1a`").

The TaxForm Class

The `TaxForm` class has the following properties and methods.

- formname — Name of the form
- title — Title of the form; used when displaying the tax form
- calculated — True if the tax form has been calculated
- is_spouses — True if the tax form belongs to the spouse
- lines — Object containing line values (see the `Line` class below)

- add() — Add line values
- calculate() — Calculate the tax form
- getHTML() — Get a block of HTML to display the tax form with its current values
- getValues()) — Copy the values from the tax form to an object of name-value pairs
- isTaxpayers() — Tax form belongs to taxpayer as opposed to the spouse
- isUsed() — Tax form has values entered; it is not empty
- line() — Return the value of a line
- max() — Like Math.max for line values
- min() — Like Math.min for line values
- putValues() — Copy name-value pairs to the tax form
- recalculate() — Calculate the tax form; ignore the calculated property
- resetCalculation() — Set the calculated property to false
- round() — Like Math.round for line values
- subtract() — Subtract line values
- toHTML() — Convert tax form to HTML
- toString() — Convert tax form to a string

The Line Class

The Line class has the following properties and methods.

- name — Line number (not used)
- label — Description of line (used for displaying in output)
- value — Value of the line (initially zero)
- min_value — Minimum allowed value (not used)
- max_value — Maximum allowed value (not used)
- user_supplied_value — True of value was entered by user (not calculated)
- initialized — Used to find values that were never initialized
- user_value — Assigned to when value is set by user supplied value

- toString() — Convert line to a string

The value of the line is stored in the `value` property, but there is also a pseudo property, `user_value`, that can be assigned to when the user has supplied the value. The difference is that when a user supplies a value, it will not be overridden when the tax form is calculated. Values read from the web page are entered using `user_value`. This allows the user to experiment with entering different values and they will not be changed if the tax calculation calculates a different value.

The TaxFormObj Module

The TaxFormObj module is used to manage all the tax form objects that have been created.

The TaxFormObj module contains the following functions.

- createForm() — Given the name of a tax form, create an instance of that tax form
- getForm() — Get a reference to an existing form

- getForms() Get references to all forms of a particular type
- getOrCreateForm() If a tax for exists, return it; otherwise create it
- getTextValue() Get the text values from a line in all forms of the same type
- getValue() Get the sum of the values from a line in all forms of the same type
- reset() Reset the global variables to their initial state

The TaxInfo Module

Most tax information is contained in the lines of the tax forms of the tax forms and can be gotten by calling `TaxFormObj.getValue()`. However, some tax values require multiple values to produce, or some values may depend on other values to determine their current meaning. The TaxInfo module provides functions that calculate these more complex tax values.

The TaxInfo module contains the following functions.

- earnedIncome() Get sum of earned income from various tax forms
- formsInPrintOrder() Get existing tax forms in the order they should be displayed
- getBusinessIncome() Get sum of income from a single business
- get1099BusinessNames() Get a list of the business names from the 1099 tax forms
- getIRAValue() Get the sum of a value from 1099-Rs that are for IRAs
- getPensionValue() Get the sum of a value from 1099-Rs that are for pensions
- getStateWithholding() Get the sum of state withholding from various tax forms
- getW2OvertimePay() Get the sum of overtime pay from W-2 forms
- getW2RetirementContributions() Get the sum of retirement contributions from W-2 forms
- getW2TipIncome() Get the sum of tip income from W-2 forms
- hasRetirementPlan() Return whether taxpayer has retirement plan at work

The Classes Module

The Classes module has a table of attributes about all the tax forms that are currently supported by the tax tools.

The Classes module contains the following functions.

- createOnDemand() Should the form be created when referenced or return zero
- findClassName() Case-insensitive class name lookup
- getClass() Return a reference to the class for the form
- isInputForm() True if form is an input form
- isOutputForm() True if form is an output form
- isSingleton() True if only one instance of the form is allowed
- isWorksheet() True if form is a worksheet
- listAllForms() Return a list of supported forms

Taxpayer

With the exception of the Social Security calculation for lump sum payments, there is only one taxpayer. The characteristics of the taxpayer and his or her spouse are stored a single instance of the Taxpayer class. During the calculation of Social Security lump sum payments, there is still only one taxpayer at a time and the tool is responsible for creating additional taxpayers and switching between taxpayers.

The Taxpayer Class

The Taxpayer class has the following properties and methods.

- filing_status Numeric value representing the filing status
- taxpayers_name Taxpayer's name
- street_address Taxpayer's street address

- city — Taxpayer's city
- state — Taxpayer's state (California)
- zip_code — Taxpayer's zip code
- taxpayers_birthday — Taxpayer's birthday (mm/dd/yyyy)
- taxpayers_age — Taxpayer's age
- is_taxpayer_blind — True if taxpayer is blind
- taxpayer_has_ssn — True if taxpayer has valid SSN
- spouses_birthday — Spouse's birthday (mm/dd/yyyy)
- spouses_age — Spouses age
- months_lived_together — Number of months lived in same home this year
- is_spouse_blind — True if spouse is blind
- spouse_has_ssn — True if spouse has valid SSN
- dependents — Array of references to dependent objects

- static getTaxpayer() — Get a reference to the current taxpayer object
- static getValues() — Copy taxpayer values from the web to an object of name-value pairs
- static putValues() — Copy initialize taxpayer from an object of name-value pairs
- static reset() — Clear the taxpayer fields on the web and remove the taxpayer object
- static resetCalculation() — Remove the taxpayer object
- static setTaxpayer() — Set the current taxpayer to a previously saved taxpayer reference
- addDependent() — Add a reference to a dependent object
- clone() — Make a shallow copy of the current taxpayer
- familySize() — Get the number of people in the taxpayer's family
- fillIn1040() — Fill in an HTML form 1040 with the taxpayer's information
- isMarried() — Return true of the taxpayer is married (filing status is MFJ or MFS)
- toString() — Convert taxpayer to string

The Dependent Class

- name — Dependent's name
- birthday — Dependent's birthday (mm/dd/yyyy)
- age — Dependent's age
- relationship — Dependent's relationship to the taxpayer
- months_lived_at_home — Number of months the dependent lived with the taxpayer this year
- gross_income — Dependents gross income
- fulltime_student — True if the dependent is a fulltime student
- mfj — True if the dependent is filing as MFJ
- dependent_supported — Dependent provided more than half of their own support
- taxpayer_supported — Taxpayer provided more than half of the dependent's support
- disabled — True if the dependent is disabled
- has-ssn — True if the dependent has an SSN
- has-itin — True if the dependent has an ITIN
- has-atin — True if the dependent has an ATIN

- isQualifiedChild() — Return true if the dependent is the taxpayer's qualifying child
- isQualifiedRelative() — Return true if the dependent is the taxpayer's qualifying relative

HTML

The HTMLBlock Class

The two JavaScript classes HTMLBlock and HTMLMgr are used to manage blocks of HTML code can be dynamically added or removed from the HTML document (i.e., the web page). The HTML block can contain any HTML code, but it must be enclosed in an HTML block element (such as , <p>, or <pre>) so the entire block can be manipulated as a single entity.

Block Names

Blocks are assigned a name, which is just a string that identifies all the blocks of the blocks of the same type. There can be more than one block with the same name so, it can be used to identify, for example, the type, category, or class, of the block.

The name must be a lowercase string that consists only of letters and numbers and begins with a letter. These restrictions facilitate parsing element IDs (see below) and allow conversion between HTML element IDs and JavaScript variable names simply by changing hyphens to underscores, and vice versa.

Unique IDs

Blocks are assigned a unique ID, which is unique among blocks with the same name. The first UID allocated is 0, then 1, then 2, and so on. UIDs are not recycled, so they remain unique the execution of the program. Since UIDs are allocated in increasing order, they can be used to determine the order blocks of the same type were created.

The block name and UID can be used to uniquely identify an HTML block among all blocks managed by the HTMLMgr module.

Field Names

The field name identifies a single field in the block, and must be unique within the block.

The name must be a lowercase string that consists only of letters, numbers, and hyphens. The field name must begin with a letter. Unlike a block name, a field name may contain hyphens.

Element IDs

The HTML elements in an HTML block and the HTML block itself need a consistent method to identify the elements so they can be managed by a common set of routines. Therefore, the ID attribute of an HTML element is used to access any element in the HTML block. Since the ID attribute of every HTML element must be unique, the element IDs of the elements in an HTML block will consist of the name of the HTML block, the unique ID of the HTML block, and the name of the field within the block separator by hyphens.

```
blockName-uid-fieldName
```

HTML blocks are used to implement the external representation of tax forms (i.e., HTML or web interface). This is where the user either enters information into the tax form or sees the results of the tax calculation.

The HTMLBlock class has the following properties and methods.

- fields — Reference to information about the accessible fields in the HTML block
- html — Reference to the block of HTML code
- name — Name of the block (usually lowercase tax form name)
- uid — Unique ID among blocks with the same name

- clearValues() — Set the value of the fields in the HTML block to ""
- getFormname() — Get the name of the form that corresponds to the HTML block
- getID() — Get the element ID of the HTML block

- getValues() Copy the field values from the HTML to an object of name-value pairs
- putValues() Copy the values from an object of name-value pairs to the HTML

The HTMLMgr Class

There is a single instance of the HTMLMgr class and it manages all the instances of HTML blocks.

- static getMgr() Get a reference to the current HTML manager object
- addBlock() Add an HTML block object to the web page
- allocateUID() Allocate a unique ID for an HTML block
- createAndAddBlock() Create an HTML block for a tax form and add it to the web page
- createBlock() Create an HTML block for a tax form
- getBlock() Return a reference to a loaded HTML block
- getBlocks() Return an array of loaded block with the specified name
- lastBlock() Get the element ID of the last loaded HTML block with same name
- removeBlock() Remove an HTML block from the web page
- removeOutputForms() Remove all HTML blocks for output tax forms from the web page
- reset() Remove HTML blocks from the web page and reset the HTMLMgr

The HTMLBuild Class

The HTMLBuild class provides methods to construct blocks of HTML code dynamically. This is currently, only used by the toHTML method of the TaxForm class. It enables the TaxForm class to create a generic HTML representation of a tax form whenever a customized version has not been provided.

- html_doc String containing the HTML document
- id A unique ID for the HTML document
- indent_level Current indentation level within the HTML document

- addElement() Add an HTML container element
- addVoidElement() Add a non-container HTML element
- putAfter() Insert the HTML document into the web page
- remove() Remove the HTML document from the web page
- startElement() Add the start of an HTML container element
- stopElement() Add the end of an HTML container element
- toString() Convert the HTML document to a string

The HTML Module

In the tax tools, almost every HTML element that is accessed (e.g., get value, put value, show, hide) is assigned an ID with the HTML ID attribute. The HTML module provides a variety of functions that facilitate accessing HTML elements using the element ID.

- addListener() Add a listener
- changeBackgroundColor() Set the background color
- changeTextColor() Set the text color
- closeAllDetails() Find and close all details HTML elements
- closeDetails() Close an HTML details element
- dispatchEvent() Trigger (dispatch) an event
- findSummary() Get the element ID for the summary element of a details element
- getCSSGlobalVariable() Get the value of a CSS global variable
- getElementValue() Get the value of an element
- getSummary() Get the value of an HTML summary element
- getUserInput() Given the element ID, get the value with some additional processing
- hideElement() Given an element ID, make sure it is not visible

- isValidElementID() Return true of the element ID refers to a valid HTML element
- openDetails() Given the element ID of an HTML details element, open it
- putElementValue() Set the value of an element
- putSummary() Set the value of an HTML summary element
- putUserOutput() Get the value of an element with some additional processing
- remove() Remove an element from the web page
- scrollTo() Scroll the web page to an element so it is visible
- setFocus() Set the focus to an element
- showElement() Make an element visible

Save and Restore

The File Module

The File module provides two functions. The `saveToFIle()` function copies a data object to a file in the user's `Download` folder and the `restoreFromFile()` function copies data from a file to a similar data object. A selection box to used locate the file to restore, but a predefined filename is used when a file is saved.

- restoreFromFile() Load the web page with values that were previously saved to a file
- saveToFile() Save the current values on the web page to a file

The SaveRestore Module

The SaveRestore module provides a common mechanism for each tax tool to allow the user to save and restore the information they have entered into the tool. The `initialize()` function establishes the connection, but there are a few dependencies that this module expects in order to work correctly. The following elements must be defined in the HTML document (i.e., web page):

- The save button must be assigned ID "save-button".
- The file selection box for the restore button must be assigned ID "input-file".
- Fields with the IDs "tax-tools-version", "title", "taxtool-container", and "error-message-output" are standard in all the tax tools and are expected to be defined.
- Tooltips for the IDs "save-button-tt" and "restore-button-tt" should be added.

The SaveRestore module provides the function:

- initialize() Initialize the Save/Restore module

ToolTips

ToolTips are short messages that pop up when you move the mouse over one a field that has a popup defined. ToolTips are implemented in two source files in the `Library` folder: `CSS/ToolTips.css` and `TAXTools/ToolTips.js`.

To add a tooltip, you need to add the following statements to the HTML file.

```
<link rel="stylesheet" href="../Library/CSS/Tooltips.css" />
<script type="module"  src="../Library/TAXTools/Tooltips.js"></script>
```

The element(s) that the tooltip applies to should be enclosed in an HTML block element. That element must be assigned the class `trigger` and assigned the `tooltipID` attribute. The tooltip ID used to identify each tooltip can be any unique string, although you can assign the same tooltip to more than one HTML element. For example:

```
<div class="trigger" tooltipID="#blah-blah-blah-ID">
      Blah blah blah.
</div>
```

The tooltip itself needs to be added as a small section of HTML code as follows. This code should be at the end of the <body>. Everything inside the element is included in the tooltip, which can include any valid HTML code.

```
<div class="tooltip" id="blah-blah-blah-ID">
      <h2>Tooltip Title</h2>
      <p>Tooltip text.</p>
</div>
```

Tooltips in the tax tools are typically collected in a separate HTML file and included in a unit at the end of the main HTML code file.

```
<?php include "tooltips.html"; ?>
```

All the tooltips in the file are enclosed in a HTML block, which is assigned the class “help-fields”. This class allows the formatting applied to HTML code in the tooltips to be defined as a subset of this class and not affect any HTML code outside this class.

```
<div class="help-fields">
</div>
```

Utilities

The Dates Module

The Dates module provides several convenience functions that are mostly tailored to the needs of tax calculations.

- getAge() — Given two dates, return the number of years separating them
- getDateObject() — Given a date or date string, return a date object
- getEndOfYearAge() — Given a birthday, return the age of the person at the end of the year
- getLastYear() — Get the last year
- getPreviousDay() — Get the previous day
- getTaxYear() — Get the tax year; if date before April 15, return last year, else this year
- getThisYear() — Get this year
- getToday() — Get date today's date formatted according to the user's locale.
- isBefore() — Compare two dates
- isEqual() — Compare two dates
- isValid() — Return if date or date string is a valid date

The Debug Module

The Debug module provides various ways of viewing the internal information and intermediate calculations of the tools.

- enter() — Log entry into a function to the debug log (i.e., not the console)
- exit() — Log exit from a function to the debug log (i.e., not the console)
- getKeywords() — Scan a string and extract debug keywords
- log() — Log a message to the debug log (i.e., not the console)
- reftrace() — Log a reference trace message to the debug log (i.e., not the console)
- reset() — Reset the debug module's global variable to their initial state.
- toString() — Convert debug information to a string
- turnOn() — Display debug information based on which keywords have been entered

The Ensure Module

The ensure module provides a variety of functions that are typically used to ensure the type of value passed as a parameter is correct. With the exception of the `processError()` function, these function all throw an exception if the expected condition is not met. The `processError()` function is expected to be called from a catch phrase to execute a common set of functions when an error occurs.

- expression() Throw exception if value evaluates to false
- isArray() Throw exception if value is not an array
- isFalse() Throw exception if value is not false
- isFilingStatus() Throw exception if value is not a valid filing status constant
- isFunction() Throw exception if value is not a function
- isNonEmptyString() Throw exception if value is not an empty string
- isNonZeroNumber() Throw exception if value is not a number or is zero
- isNumber() Throw exception if value is not a number
- isObject() Throw exception if value is not an object
- isString() Throw exception if value is not a string
- isTaxpayer() Throw exception if value is not a valid taxpayer constant
- isTrue() Throw exception if value is not true
- isType() Throw exception if value is not the specified type
- isValidDate() Throw exception if value is not a valid date or date string
- isValidElementID() Throw exception if value is not a valid HTML element ID
- isValidNumber() Throw exception if value is not a valid number
- processError() Standard processing when an error is caught

The Eval Module

The eval module contains only one function, which is a single mathematical expression evaluator. The expression evaluator is used in the Simpler Math Calculator tool, and it is also used to evaluate every numeric input value for all the tax tools.

- expression() Evaluate and expression

The IncludeFile Module

The HTML language desperately needs a mechanism to include other files. Because that is not available, the tax tools use two mechanisms to provide this functionality. Most files are included using the include capability in the PHP language; however, there are a few places where the tax tools use the include capability provided by the `loadIncluded()` function in this module. This function allows an HTML file to include another file using a statement like this:

```
<div include-file="filename"> Missing IncludeFile.js </div>
```

The way this works is the function scans the document for HTML elements with the attribute "include-file". Any element that is found is replaced with the file named as the value of the attribute.

Unfortunately, both mechanisms require an HTML file to be loaded from a server, not by double clicking on the file on the local system. The advantage of using the include mechanism provided by this module over the PHP mechanism is that you can name the source file with the "html" suffix. The included files will not be included, but the rest of the file will be displayed.

- loadIncludes() Load an include file from an HTML file

The Num Module

The Num module provides several convenience functions that are mostly tailored to specific needs in one or more tax tools.

- format() Convert a number to a comma separated string

- isNum() Return true of the value is a valid, finite number
- limit() Limit a number to a range of values
- round2() Round a number to 2 decimal places
- roundDown() Round a number down to an even multiple of an index
- roundup() Round a number up to an even multiple of an index
- toInteger() Convert string to number; evaluate expressions, ignore commas and $

The Objects Module

The Objects module provides several convenience functions that are mostly tailored to reducing unnecessary output when displaying the information in an object.

- isEmpty() Return true if object contains only properties set to 0 or ""
- isUsed() Return true if object contains properties not set to 0 or ""
- removeUnused() Return object but filter out properties that are set to 0 or ""
- toString() Convert object to string; name-value pairs with spacing between

The SalesTaxFromCDTFA Module

Sales tax varies from county to county and the current rate of sales tax is needed to calculate the sales tax deduction when using itemized deductions. The tax tools that use this information retrieve it from an external site. The `fetchSalesTaxRate()` function in this module is used to get that information. The web site being used only provides sales tax for counties in California, so the tax tools can only calculate the sales tax deduction for California taxpayers.

- fetchSalesTaxRate() Lookup sales tax from external web site (California only)

The Str Module

The Str module provides a variety of convenience functions for manipulating strings. Some of these function are generic, but many of these functions are needed to format the information that is presented to the user.

- camelCaseToEnglish() Convert string from camel case to standard English
- camelToSnakeCase() Convert string from camel case to snake case
- caseEqual() Case-insensitive comparison of two strings
- clean() Remove unnecessary whitespace
- downshift() Convert to lower case
- empty() Return true of the string is empty
- equal() Compare two strings
- kebabToSnakeCase() Convert string from kebab case to snake case
- prefixLines() Insert prefix to lines separated by newlines within a string.
- snakeCaseToEnglish() Convert string from snake case to standard English
- snakeToCamelCase() Convert string from snake case to camel case
- snakeToKebabCase() Convert string from snake case to kebab case
- upshift() Convert to upper case
- upshiftFirst() Upshift the first character in the string
- wrap() Wrap string into multiple lines by breaking on word boundaries
- wrapLines() Wrap lines in a string with multiple lines separated by newlines

Tax Tools

Age Calculator

The age calculator waits for one of the inputs to change. Depending upon which value changes, it uses that value and one of the other values to calculate the third value. Several tax calculations need to know a person's age at the end of the tax year, so in addition to this tool, several of the tools use this calculation internally. That makes this tool useful for testing that code.

Simple Math Calculator

This is a simple four-function mathematical calculator. Calculations on tax forms do not require more advanced mathematical functions so this can be a useful aide when working with tax forms. Also, the input fields in all the tax tools allow you to enter an arithmetic expression in place of a numeric value, and the same expression evaluator is used in both cases. That makes this calculator useful for testing the expression evaluator.

Tax Return Calculator

The Tax Return Calculator tool is unlike the other tax tools. Rather than providing a simplified abstraction for part of the tax calculation, the Tax Return Calculator uses actual tax forms for the input information and the result is similar to an actual tax return.

Other Tax Tools

The remainder of the tax tools (those not described above) are all variations on a theme:

- Each tool provides a web page where you enter information. It only asks for the information it needs to perform the calculation the tool is designed to perform.
- It maps the input information to fields on actual tax forms and creates digital tax forms internally.
- It performs the calculation implemented by each of the tax forms.
- It gets the information it needs from the appropriate tax form and writes the result of the calculation to the web page for the user to see.

The tax tools that use this basic design are:

- Alternative Minimum Tax Calculator
- California Sales Tax Calculator
- California Sales Tax Deduction Calculator
- Federal Estimated Tax Calculator
- California Estimated Tax Calculator
- Required Minimum Distribution Calculator
- Senior Deduction Calculator
- Small Business Tax Calculator
- State Income Tax Refund Calculator - Since the amount of income tax paid to the state is deductible on the federal tax return, a state tax refund may be taxable as income on the following year's federal tax return. The calculation to determine how much of the refund is taxable is described in the instructions for form 1040. See the section for Schedule 1, Additional Income, Line 1, State and Local Income Tax Refund Worksheet.
- Taxable Social Security Benefit Calculator – The amount of the taxpayer's Social Security benefit that is taxable is a common question and it is difficult to predict beyond 0-85%. This tool performs the calculation based on the information provided by the user. Since the user will not

know the exact values needed for the calculation, this tool can only provide an estimate. The worksheet to perform the calculation is described in the instructions for form 1040 in the instructions for entering lines 6a and 6b.

- Lump Sum Social Security Tax Calculator

Software Conventions

Coding Conventions

- Indentation in HTML, CSS and JavaScript files is 1 tab and tabs are 4 spaces. UNIX style newlines are used between lines, not Windows style carriage return- newline combinations.
- Lines are a maximum of 93 characters to prevent wrap-around when printed.
- Use `===` and `!==`, not `==` and `!=`
- Do not use var.

Naming Conventions

A variety of naming conventions are used to create symbolic names beyond the rules implied by the languages. When the name used all lowercase letters and underscores are used to separate words or parts, that is called “snake case”. “Kebab case” is similar except it uses hyphens to separate works. “Camel case” is when there is no separation between words, but each word is capitalized. The following list shows the naming conventions used in the tax tools source code.

Variable names	Snake case
Object Key Names	Snake Case
Procedure and method names	Camel case, 1st letter is lowercase
Class names	Camel case, 1st letter is uppercase
CSS class names	Kebab case
HTML element IDs	Kebab case

Tax information is entered and displayed on web pages and it is also stored internally in objects. This necessitates moving data between HTML and JavaScript. While this is not difficult, in order to provide tools for automating the process, the tax tools use a naming convention for tax information that allows the ID of an element on a web page to be converted to its corresponding variable name, and vice-versa. HTML element IDs use kebab-case, which uses lower case letters and separates works with hyphens. The corresponding variable name will be the same, except the hyphens are replaced with underscores.

The tax form line number corresponding to each field on a tax form is embedded in the element ID and variable name used to access that field.

Design Notes

- To make web page print-friendly, you can add the following line to your stylesheet. This rule automatically strips extra padding and background colors when the user presses Ctrl+P / Cmd+P to save it as a crisp PDF.

```
        @media print { body { padding: 0; background: none; } }
Or,
        @media print {
               body { background: none; padding: 0; }
               .form-1040 { border: none; width: 100%; }
        }
```

The TaxFormObj.js module manages the forms that are in use by the tool. Any form that is created should be created by calling `TaxFormObj.createForm()`, not by calling `new FormName()`. This allows the Forms module to keep tract of which forms have been created and providing convenient access to the form information.

Normally, a tax form must be created and calculated before values can be read from the form for use by another form, but to prevent creating and printing a lot of unused forms, if the form is just going to return 0, it does not need to be created. For example, form 1040 needs to read the withholding information from lots of forms (e.g.,W-2s, 1099s), it is not necessary to create these forms just to return 0. Any form that does not reference another form and has not already been created with initial values, does not need to be created when it is referenced.

A form does not need to be created to be accessed. If the form does not exist, the value returned will be 0. This is sufficient for any form that does not access another form such as the W-2 or 1099-INT. If a form does exist, its `calculate()` method must be called to make sure the data is correct (the modified flag can be used to prevent recalculating a form when it is not necessary).

A form can be accessed by `TaxFormObj.getValue("formname", "lineno")`. This will create the form if it does not exist, and it will calculate it if it has not been calculated. This is how most forms are accessed, and it automatically creates a dependency chain. If a form needs a value from another form, that form will be created and calculated before the value is returned.

Source Control

The project name, the repository name, the folder name that contains the source code, and the nickname for the Tax Analysis and Experimentation Tools are all "TAXTools".

Source code version control for the tax tools is implemented using Git and GitHub. Git is a distributed version control system originally developed for the Linux kernel, but can be use for any project where versioning changes to files is necessary. It is particularly useful in situations where there are multiple people making changes in separate locations. For more information on Git, go to their web page at git-scm.com.

While Git is designed to be a distributed source control system, it is very common to have a central repository that all the developers draw from. GitHub fills this need. It provides a common reference copy of the source code that is always available on the web and always backed up. GitHub provides both free and fee-based hosting for Git repositories. For more information on GitHub, go to their web page at github.com.

Repositories

A repository is a directory that contains the source code for the project and has been initialized for use with Git. When the directory is initialized, Git creates a subdirectory named "`.git`" at the top of the source directory. This is where Git stores all the information it needs to track the changes to the source code.

Since Git is a distributed source management system, a project can have any number of repositories. Typically, each developer has a copy on their system. In addition, there may be one repository that is considered the "main" repository where changes are collected and other repositories can be synchronized to keep up with the changes that have been made by others.

This is the case for the tax tools. The main repository is hosted on GitHub at `github.com/bb3413/TAXTools.git`, and each developer has a local copy of the repository. To work on the tax tools, you start by cloning the repository on GitHub, which creates a local copy that you can use to do your work. By keeping this local copy synchronized with the main repository stored in GitHub,

you're able to get the latest changes from the main repository and push your updates back to the main repository so others can use them. To "clone" of the official repository, use the following command.

```
$ git clone https://github.com/bb3413/TAXTools.git
```

Authentication

When working with remote systems, it may be necessary to provide some form of authentication. The tax tools repository is a public repository, so you can read from the repository (for example, clone) without providing any authentication, but to write to the repository, you will need to be authenticated in one of two ways.

1. You can access the tax tools repository using the following HTTPS URL.

   ```
   https://github.com/bb3413/TAXTools.git
   ```

   If you use this URL to write to the repository (for example, `git push`), you will have to provide the account name and a security token each time you write to the repository. This may work well when working from Windows, since the Windows Credential Manager will cache this information and supply it on subsequent commands. See the next section for the steps to create a security token.

2. Alternatively, you can use the secure shell (SSH) to access the repository. In this case, you use the following Git address.

   ```
   git@github.com:bb3413/TAXTools.git
   ```

   This may be more convenient when working from the command line (for example, the Bash shell). However, to use this address, you need to setup the SSH connection as follows.

   a. Execute the `ssh-keygen` command and press enter to accept the default for all the questions. This will create a public (`id_ed25519.pub`) and private key (`id_ed25519`) in the directory `$HOME/.ssh`.

      ```
      $ ssh-keygen -t ed25519 -C "comment"
      $ cat $HOME/.ssh/id_ ed25519.pub  # Show the public key
      ```

   b. Then, you can copy and paste the public key to GitHub. To enter the public key on GitHub, go to Settings->SSH and GPG keys->New SSH key.

   The first time you use this address, you will get a warning asking if you trust the remote system, but you will not need to provide any authentication information.

When reading or writing to the repository, you can use either form of address. It only makes a difference in how you are authenticated.

In addition to providing authentication, you need to be added as a "collaborator" on the project by the owner of the repository if you want to make change to the project.

Creating a Token

To create a personal access token on GitHub, follow these steps:

- In a web browser, go to github.com.
- Log into account bb3413. Use password for account bb3413.
- Go to Home->Settings->Account->Developer settings->Personal access tokens.
- Select Tokens (classic)
- Click on Generate new token
- Select Generate new token (classic)
  - Enter the account password.
  - Note: Full CLI access to repositories. Use web for everything else.

- Expiration: 90 days
- Click on Generate token

Branches

A branch refers to a separate version of the same code. The original version continues in one branch and a separate branch is created to keep copies of changes that deviate from the original branch. Creating a branch allows developers to work on changes without affecting each other. When a change is complete, the branches can be merged back together and any conflicts can be resolved.

The tax tools repository contains two branches. The main branch, named “main”, contains the currently released version of the tax tools. Changes to the main branch may be made for small fixes, but generally, the main branch is only changed by periodically merging with the development branch.

The other branch is the development branch. It is named “develop”, and it contains the new features and fixes. Once a change is believed to work, it can be added to the development branch to give it more exposure and verify that it is stable. Merging the development branch with the main branch is based on the need for the changes and the perceived stability of the development branch.

Small changes may be made directly in the development branch, but new features should be developed in their own branch off of the development branch. Developing new features in their own branch allows the changes to be checked in (committed) even if they are not complete. The need to check in incomplete changes prevents the need to have “stashes” for each change that is in progress but not complete (stashes are needed to save files that are not checked-in when you need to change to another branch to work on another problem).

Create the Initial Repository

Initially, the tax tools consisted of a local directory that contained all of the source code for the tools. This section describes the steps that were used to create the main repository on GitHub from the original source code. This is for information only since the tax tools have already been set up to use Git and GitHub to manage the source code.

1. Create a personal access token on GitHub. GitHub requires using a personal access token instead of a password for authentication when using the command line (e.g., `git push` command).
    - In a web browser, go to github.com.
    - Log into account bb3413. Use password for account bb3413.
    - Go to Home->Settings->Account->Developer settings->Personal access tokens.
    - Select Tokens (classic)
    - Click on Generate new token
    - Select Generate new token (classic)
        - Enter the account password.
        - Note: Full CLI access to repositories. Use web for everything else.
        - Expiration: 90 days
        - Click on Generate token

2. Create the new, empty repository on GitHub.
    - In a web browser, go to github.com.
    - Log into account bb3413. Use password for account bb3413.
    - Click on the "+" icon in the top-right corner and select "New repository."
        - Repository Name: TAXTools
        - Description: Tools for estimating income tax.
        - Choose Visibility: Public
        - Add README: Off
        - Add .gitignore: No .gitignore

- Add license: No license
- Click "Create repository."

3. Convert the source code directory to a Git repository. The `git init` command will convert the current directory into a Git repository where you can use Git commands to track changes to files in the directory. A Git repository has a subdirectory at the top of the source directory named `.git`. This is where Git stores all the information it needs to track changes to the files.

```
$ cd TAXToolsDirectory
$ git init
```

There may be some files in the directory that are not part of the source code for the project (for example, object files generated by a compiler or files containing private information) and, therefore, you want Git to ignore those files. You can specify these files by making an entry for them in the file `.gitignore`, which you put at the top of the repository. The Git ignore file contains information that is needed by all users of the repository so it should be added to the repository just like a source file. Note: it should only identify temporary files; files that are not ready for release should be left in the source tree so they can be shared when someone clones the repository.

```
$ cat <<-EOF >>.gitignore
       # DreamWeaver status files
       _notes/

       # Editor temporary files
       ~*
       *.swp

       # Modules loaded with NPM
       nodes_modules/

       # NPM status files
       package.json
       package-lock.json

       # ESLint configuration file
       eslint.config.mjs
EOF
```

4. Add the source code files to the repository. The `git add` command adds all the files in the repository to the staging area. Commit the changes in the staging area to the repository. At this point, the source code directory is a Git repository and changes to the files in the directory will be tracked by Git.

```
$ git add .
$ git commit –m "Initial commit."
```

5. The main branch is named "master" by default. Change it to "main".

```
$ git branch -M main
```

6. Mark the current version of the tax tools in the main branch with the release number.

```
$ git tag VersionNumber        # e.g., "v2025.08"
```

7. Connect the local repository to the remote repository. The `git remote add` command allows you to specify a name and URL for a remote repository. The name "origin" is the default name used by Git when the name of the remote repository is not specified.

```
$ git remote add origin git@github.com:bb3413/TAXTools.git
$ git remote add originSSH git@github.com:bb3413/TAXTools.git
```

```
$ git remote add originHTML https://github.com/bb3413/TAXTools.git
```

8. Populate the remote repository with the files from the local repository. The `-u` option sets "origin" and "main" as the default remote repository and branch. This allows you to later execute the `git push` and `git pull` commands without having to specify these parameters.

```
$ git push -u origin main
$ git push origin --tags
```

9. Create the development branch.

```
$ git branch develop
$ git switch develop
```

10. Update the version information.

```
$ edit Version.js Version.html
$ git add .
$ git commit -m "Create new development version."
```

11. Push the development branch to GitGub. The `-u` option changes the default branch, set above, to now be the "develop" branch. The origin is still set to the tax tools repository on GitHub.

```
$ git push -u origin develop
```

12. Go back to GitHub, refresh the page, and make sure the repository looks correct.

Using Git with the Tax Tools

Configuration Settings

Git configuration settings can be set for a user or a repository. User settings apply to the user whenever they are using Git regardless of which repository they are in. Repository settings apply to all users in a specific repository.

The `git config` command allows you to set Git configuration values. If the `--global` option is used, the command sets the value for the user. If the `--global` option is not used, the command sets the value for the current repository.

User configuration settings are stored in the file `$HOME/.gitconfig`, and repository configuration settings are stored in the file `.git/config` at the top of the repository.

User Settings

Before using Git to make changes, you need to set your name and email address in Git. This information is needed to track changes to the files.

```
$ git config --global user.name "First Last"
$ git config --global user.email "Email address"
```

The default name for the initial branch when creating a Git repository is "master". This term is deemed improper so it is recommended that you change the default to "main" for any new Git repositories.

```
$ git config --global init.defaultBranch main
```

When a Git repository is not stored on a local disk (e.g., cloud storage), you need to tell Git that you trust the directory containing the repository.

```
$ git config --global --add safe.directory \
       '/mnt/g/My Drive/Documents/VSCode/TAXToolsDev'
```

Create Local Repository

Since the tax tools are available on GitHub, you can make a local copy of the repository (i.e., source code + Git tracking information) by changing directories to where you want to save the repository and execute the following Git command to clone the repository. This will create a subdirectory named "TAXTools" with the most recent version of the source code for the tax tools.

```
$ cd directory
$ git clone https://github.com/bb3413/TAXTools.git
```

Create Account on GitHub

Once you have a local copy of the tax tools repository, you can make changes to the files and perform most of the steps described in this section, but you will not be able push your changes to the remote repository so others can see them until you have a GitHub account and have been added as a "collaborator" on the repository on GitHub.

1. Create a GitHub account
2. Ask the owner of the repository on GitHub to be added as a collaborator. The repository owner will need your GitHub account name. The owner can add your account as a collaborator by going to the repository and selecting Settings->Collaborators->Add people.

Synchronize with Remote Repository

Before you make a change to the tax tools source code, you want to make sure your local repository contains the most up to date version of the source code file. To synchronize your local repository with the files on the remote repository, use the `git pull` command.

```
$ git switch branch          # Make sure you are in the correct branch
$ git pull
```

Get Status

There are several commands to get information about the repository.

```
$ git status        # Information about the working directory.
$ git show          # List the changes to the current branch.
$ git log           # Show the commit history on current branch.
$ git diff          # See what changed in a file.
```

Example

You can use `git status` to see which files in the working directory have changes. This command can be executed from anywhere in the working directory.

```
$ cd $TAXTOOLS
$ git status
```

From the files that are listed, if they are already being tracked by Git, then you can use the `git diff` command to see which lines in the file have changed.

```
$ cd directory
$ git diff filename
```

Once you are satisfied with the changes, you can add them to the staging area and commit the changes to the repository. When specifying "." as the files to add, that only adds the file from the current directory, so change directories back to the top of the working directory to add all the changes that have not been staged.

```
$ cd $TAXTOOLS
$ git add .
$ git commit -m "Description of change."
```

Or, you can do it one step.

```
$ git commit -a -m "Description of change."
```

Sync with GitHub.

```
$ git push
```

Select a Branch

If you have added, deleted, or modified any files but have not committed those changes, they will remain in your source directory even if you change to a different branch. For this reason, it is important to make sure you there are no changes that have not been committed before you change branches.

```
$ git status        # Check for changes that have not been committed.
```

After you switch to a new branch, it is also good to make sure the branch is up to date with any changes that have been posted to the remote repository.

```
$ git switch branch
$ git pull          # Make sure it is up to date.
```

Below are some other useful commands for use with branches. The functionality of the `get switch` command is duplicated by the `git checkout` command. The `git checkout` command can switch branches, restore files, and check out commits. The `git switch` command is a newer command intended specifically for switching branches.

```
$ git branch               # List all branches
$ git branch branch        # Create new branch; does not switch to it
$ git branch -d branch     # Delete branch
$ git branch -r            # List all branches in remote repository
$ git checkout branch      # Switch to branch
$ git checkout -b branch   # Create new branch and switch to it
$ git switch branch        # Switch to branch
$ git switch -c branch     # Create new branch and switch to it
```

Create a New Branch

For most changes, you will start at the development branch and create a new branch where you will make your changes. Before you create a new branch, make sure you are in the correct branch.

You can create a new branch and switch to it in two steps.

```
$ git branch branch        # Create new branch
$ git switch branch
```

Or, one step.

```
$ git checkout -b branch   # Create new branch and switch to it
```

Set the default location for `git pull`.

```
$ git branch -set-upstream-to=develop branch
```

Set the default behavior for handling merge conflicts when you do a `git pull`.

```
$ git config pull.rebase false
```

Add a New File

To add a new file to the project, you need to create the file, add it to the staging area, and commit the change (i.e., whatever is in the staging area).

1. Before you add a new file, make sure you are in the correct branch and the correct location, then create the file.
2. Add the new file to the staging area.

   ```
   $ git add filename
   ```

3. Commit the changes in the staging area. This will commit the changes in the staging area (i.e., the new file) to the repository. At this point, the new file has been added to the current branch. At some point, you may want to merge the current branch into the development branch.

   ```
   $ git commit –m "Description of change."
   ```

Delete a file

The `git rm` command deletes a file from the working directory and adds the change to the staging area, so after executing the `git rm` command, you can commit the change.

```
$ git rm filename
$ git commit –m "description"
```

Change a File

To change file to the project, you need to edit the file, add it to the staging, and commit the change (i.e., whatever is in the staging area).

1. Before you change a file, make sure you are in the correct branch and that the branch is up to date. Then, change the file.
2. Add the file you changed to the staging area.

   ```
   $ git add filename
   ```

3. Commit the changes in the staging area. This will commit the changes in the staging area (i.e., the modified file) to the repository. At this point, the modified file has been changed in the current branch. At some point, you may want to merge the current branch into the development branch.

   ```
   $ git commit –m "Description of change."
   ```

Revert a Change

To discard a change to a file in the working directory before it has been staged use the `git restore` command. This restores the file to the state it was in after the last time it was committed.

```
$ git restore filename        # Restore the file
$ git restore .               # Restore all files in the current directory
                              # and subdirectories.
```

Stage a File

Any change to the working directory, such as adding, deleting, or modifying a file, needs to be added to the staging area before the change can be committed. Use the `git add` command to stage a file. If more than one file is changed, you can specify all the names to the `git add` command or you can specify dot (".") instead of the file name to add all files that have been added, deleted, or modified anywhere in the current directory to the staging area.

```
$ git add filename
```

If you stage a file and then make another change to the file, you need to re-stage the file; otherwise, only the first change will be committed, but the second change will remain in the file in the working directory.

Instead of the `git add` command, you could also use the `git stage` command. These commands can be used interchangeably.

Revert a Stage

If you make a change and add it to the staging area, but you have not committed the change, you can remove the file from the staging area with the following command. The file in the working directory is not affected.

```
$ git reset filename          # Unstage a file
$ git reset                   # Unstage everything
```

Commit a Change

The `git commit` command will make all the changes in the staging area a permanent change in the repository.

```
$ git commit –m "Description of change."
```

Revert a Commit

```
$ git revert HEAD          # Revert the last commit
```

Push Changes to the Remote Repository

The repository on GitHub is considered the master repository. While Git is capable of distributed source management, it is common for a central repository to be used as a collection point for all changes and for making official releases. For the tax tools, the repository on GitHub is that repository. The "local repository" refers to the copy of the repository that you have on your local system, and the "remote repository" refers to the repository on GitHub.

If you have permission to change the remote repository you can use the `git push` command to push committed changes from the local repository to the remote repository.

```
$ git push
```

Make a Change to Development Branch

1. Create a local repository if necessary.
2. Bring the local repository up to date from the master repository on GitHub.

   ```
   $ cd TAXToolsDirectory
   $ git pull
   ```

3. Create a branch off the development branch for the change.

   ```
   $ git checkout develop            # Switch to the development branch
   $ git checkout –b branch          # Create the new branch
   ```

4. Feature development loop.

   Edit the files. Edit the version number and version log

   ```
   $ code .      # Run VS Code editor in the current directory.
   ```

   Commit the changes.

   ```
   $ git add .   # Add the changes to the staging area.
   $ git diff –staged
   $ git commit –m "Description of change"
   ```

5. Merge the changes into the development branch

   ```
   $ git checkout develop     # Change back to the development branch
   $ git merge branch         # Merge the changes
   ```

6. Update Version Information. Edit the version files directly in the development branch. Increment version number (third number) and update version log.

```
$ edit version.js version.html
$ commit –a "Update version"
```

7. Update the master repository on GitHub.

```
$ git push
```

8. Update the Web page with the new prototype version. Open DreamWeaver and push the folder with the tax tools to the web.

Make Change to Main Branch

1. Make sure there are no changes files in local repository. The `git status` command shows which files have been added, deleted, or modified in the local repository.

```
$ git status
```

2. Change to the release branch (main).

```
$ git checkout main
$ git pull
```

3. Edit the files directly in the main branch.

```
$ edit files
```

4. Update version information. Increment the version number (second number) and update version log.

```
$ edit version.js version.html
```

5. Commit the changes in the modified files to the repository.

```
$ git commit –a "Description of change."
```

9. Update the master repository on GitHub.

```
$ git push
```

Merge Development Branch into Main Branch

Merging the development branch into the main branch creates a new release of the tax tools. The head of the main branch is always the current release of the tax tools.

1. Change to the development branch (develop).

```
$ git checkout develop
$ git pull                 # Make sure the branch is up to date.
```

2. Make sure the PDF files in the Documents directory are up to date.

3. Make sure there are no edited files in the working directory. The `git status` command shows which files have been added, deleted, or modified in the working directory.

```
$ git status

# If necessary, check in any changed files.
$ git commit –a –m "Description of changes."
$ git push
```

4. Update the version information. Set the version number for the next release on the main branch and make an entry in the log file.

```
$ cd Version
$ edit version.js version.html
```

```
$ git commit –a –m "Update version information."      # Add and commit
$ cd ..
$ git push
```

5. Change to the release branch (main).

```
$ git checkout main
$ git pull                 # Make sure the branch is up to date.
```

6. Merge the development branch.

```
$ git merge develop
$ git commit –a –m "Merged from develop branch."
```

7. Create a Release Tag.

```
$ git tag VersionNumber       # e.g., "v2026.00"
$ git push                    # Push the commits
$ git push --tags             # Push the tags
```

8. Copy the release files to the release directory for pushing to the web page. The current branch should still be in release branch (main).

```
$ cd $TAXTOOLS/..
$ rm –rf TAXToolsNew
$ ./Tools/UpdateRelease                  # Copy TAXToolsDev to TAXToolsNew
$ dircmp TAXTools TAXToolsNew            # Compare with previous release
```

9. If everything looks good, make a backup copy and replace the TAXTools directory with TAXToolsNew.

```
$ mv TAXToolsNew TAXTools-VersionNumber
$ tar –cvzf TAXTools-VersionNumber.tgz TAXTools-VersionNumber
$ cp TAXTools-VersionNumber.tgz $ARCHIVE/TAXTools
$ rm –rf TAXTools                        # Remove old release files
$ mv TAXTools-VersionNumber TAXTools     # Replace with new files
```

10. Update the web with the new version of TAXTools.

```
$ Open DreamWeaver
      # Copy the new Tax Analysis Tools to the web
```

11. Change the local repository back to the development branch.

```
$ cd $TAXTOOLS
$ git checkout develop
$ git pull    # Make sure it is up to date.
```

12. Update Version information for the current development release.

```
$ edit version.js version.html
$ git commit –a "Start new development version."
$ git push
```

Get a Past Version of the Tax Tools

Create a Tag

You create a tag with the `git tag` command. When you push commits to the remote repository, it does not push the tags; they must be pushed in a separate step.

```
$ git tag VersionNumber       # e.g., "v2025.08"
$ git push                    # Push the commits
$ git push --tags             # Push the tags
```

You delete a tag with the `git tag` command as well.

```
$ git tag --delete tagName  # e.g., "v2025.08"
$ git push origin --delete tagName
```

Git Help

The following two URLs show the online documentation for Git. The first URL is the online manual page for Git and the second URL is the complete Git documentation.

https://git.github.io/htmldocs/git.html
https://git-scm.com/docs

The Git command also has all of the documentation built into the command so you can access it from the command line without opening a browser.

To show the command line syntax and a list of common Git commands:

```
$ git help
```

To list all of the Git commands:

```
$ git help -a
```

To list guides or tutorials on various topics:

```
$ git help -g
```

For help with a specific command:

```
$ git help command
```

Git Terminology

Commit	Make the changes in the staging area permanent.
Head	Is the most recent commit in the current branch. Since each branch has a head, HEAD in all capital letters refers to the head of the current branch.
Local	Or local repository, refers to the developer's copy of the repository on their local system.
Origin	Is the default name Git uses to refer to a remote repository.
Remote	Refers to the remote repository, which is the same repository on a server somewhere. In our case, it is the tax tools repository on GitHub.
Repository	Is any directory that has been initialized by Git using the `git init` command. Once initialized, the directory will have a subdirectory named `.git` where Git stores all the information it needs to track changes to the files in the directory.
Staging Area	Also called the *index* or the *cache*. It is a temporary area where you "stage" changes to be included in the next commit.
Working Directory	The directory containing the source files for the branch you are currently working on. Thus, it is the general term for the source code directory where you do your work. It is also the current branch in the local Git repository.

Editing Tools

VS Code

Visual Studio Code, usually called VS Code, is a popular, free editor for developers. There are many extensions that give it additional capabilities and interoperability with other tools, but it comes with build in support for HTML, CSS, and JavaScript. For more information on VS Code, go to https://code.visualstudio.com.

Installation

1. Go to https://code.visualstudio.com/download
2. Click on the correct install option (for Windows, all the options result in the same installer).
3. Click on the enormous VS Code button to download the installer.
4. Execute the installer
    - Check **Add to PATH**. This is needed to run VS Code from the command line by entering `code`.
    - Check **Add Open with Code action to Windows Explorer file context menu**. This allows you to right click on a file to open the file with VS Code.
    - Check **Add Open with Code action to Windows Explorer directory context menu**. This allows you to right click on a directory to open it with VS Code.
    - Click **Register Code as an editor for supported file types**. This makes VS Code the default editor for the supported file types.

Configuration Settings

Synchronize your configuration settings across all the systems you use.

1. Click on the gear icon and select **Backup and Sync Settings**.
2. You will be prompted to sign in to a GitHub or Microsoft account. Select **GitHub**.

User Settings

User settings are only for the user, but apply in all projects that the user works on. User settings are stored in the file VS Code settings are stored in the file `C:/Users/`*UserName*`/AppData/Roaming/Code/User/settings.json`.

Editor	Bracket Pair Colorization	?
Terminal	Default Profile Windows	Ubuntu (WSL)
Workbench	Color Theme	Light Modern

Workspace Settings

Workspace settings apply to all users working in the workspace. Workspace settings are stored in the file `.vscode/settings.json`. at the top of the project directory. This is a workspace file are should be checked into the Git repository.

Editor	Format On Save	False	
Editor	Tabsize	4	# Spaces per indentation level
Editor	Insert Spaces	false	# Use tabs for indentation
Editor	Detect Indentation	true	# Match existing indentation
Files	Insert Final Newline	True	

Files	Trim Trailing Whitespace	True

Extensions

I use the following extensions. VS Code does not support printing, so you need to add an extension to provide the functionality. The Print extension adds print icons to the top banner of the editor panel. The ESLint and Prettier extensions are discussed below.

Print
Prettier - Code Formatter
ESLint

The following extensions may be useful; I haven't decided.

GitLens
Code Spell Checker
Jest
GitHub Copilot
GitHub Copilot Chat
Open in Browser (by TechER)

Command Summary

VS Code Window

Ctrl + Shift + P	Command palette
Ctrl + ,	Open settings
Ctrl + W	Close tab
Ctrl + B	Show/hide left bar
Ctrl + J	Show/hide the panel (bottom of editor window)
Ctrl + `	Show/hide terminal
Ctrl + Alt + I	Show/hide Copilot chat window
F11	Toggle full screen

Search and Replace

CTRK + Shift + R	Find text across all files
CTRL + P	Find and open file
CTRL + T	Go to symbol
F12	Go to definition
ALT + F12	Peek at definition
Shift + F12	Find where symbol is used
ALT + Left Arrow	Go back (like browser history)
ALT + Right arrow	Go forward (like browser history)

File Management

Ctrl + S	Save file
Ctrl + K S	Save all

Editing

CTRL + C	Copy
CTRL + V	Paste
CTRL + X	Cut (copy and delete)
CTRL + /	Toggle comment/uncomment current line
ALT + Up arrow	Move current line up 1 line
ALT + Down arrow	Move current line down 1 line
CTRL + Shift + K	Delete current line

CTRL + Z	Undo
CTRL + Y	Redo
Shift + Alt + F	Format file
Ctrl + G	Go to line

Copilot

Tab	Accept inline code suggestion
Esc	Dismiss inline code suggestion
ALT +]	Show next suggestions
ALT + [	Show previous suggestion
ALT + \	Triggers a suggestion
Ctrl + Enter	Generate up to 10 suggestions in a separate pane

Node and NPM

Note: the tools described in this section do not work when the repository is on cloud storage (e.g., Google Drive).

For now, the following steps should be done in your local repository and not checked into Git.

The Node Package Manager (NPM) is a program that lets you download JavaScript libraries from the NPM registry and use them in JavaScript projects. NPM is bundled with Node, so NPM will be installed when you install Node. Node, or Node.js, is a JavaScript runtime environment and web server, which allows you to test server-side JavaScript code on your local system.

Installation

1. Go to https://nodejs.org
2. Execute the installer
3. In a terminal window, initialize the package manager (NPM) for use with your project. This step only needs to be done once, so this step has already been done. This step creates the file `package.json`, which NPM uses to track which packages have been installed and their dependencies.

   ```
   $ cd TAXToolsDirectory
   $ npm init -y
   ```

4. Other NPM commands:

   ```
   $ npm install packageName
   $ npm list
   $ npm outdated      # List packages that need to be updated
   $ npm uninstall packageName
   $ npm update [packageName]
   ```

Prettier

Prettier - Code Formatter is the code formatter used by the tax tools.

1. Install the prettier package. The `save-dev` option indicates that the package is only used during development and will not be distributed with the project.

   ```
   $ cd TAXToolsDirectory
   $ npm install prettier --save-dev
   ```

2. Run Prettier on a file.

   ```
   $ npx prettier filename
   ```

3. In VS Code, install the extension Prettier – Code Formatter and change the following configuration settings. Restart VS Code. Use the keyboard shortcut Shift + Alt + F to format the current document and use Ctrl + Z to undo the change.

Editor	Format On Save	false
Editor	Default Formatter	Prettier – Code Formatter

ESLint

NOTE: This does not work when the source code is hosted on Google Drive. Clone a new repository on the local system and use ESLint there.

ESLint is the code checker used for the tax tools.

1. Install the ESLint package. The `save-dev` option indicates that the package is only used during development and will not be distributed with the project.

```
$ cd TAXToolsDirectory
$ npm install eslint --save-dev
```

2. Create a default configuration for ESLint.

```
$ npx eslint --init
```

3. Run ESLint on a file.

```
$ npx eslint filename
```

4. In VS Code, install the extension ESLint. Restart VS Code.

Git

Git and GitHub are the source code version control tools used for the tax tools. This section describes the installation of Git, but there is a complete discussion on how these tools are used with the tax tools earlier in this document.

Installation

1. Go to https://git-scm.com/install/windows.
2. Select the Windows tab.
3. Click on **Click here to download**…
4. Execute the installer
    - Git will be installed at: C:/Users/<UserName>/AppData/Local/Programs/Git. You may need to know this if one of the tools is not able to find the Git commands.